



RabbitMQ + Hosted Service Integration

Step-by-Step Training Guide — EmployeeApp (.NET 10)

Framework: .NET 10 | **Broker:** RabbitMQ | **Pattern:** Publisher / Consumer



Table of Contents

1. Key Concepts Explained
2. The Demo Scenario
3. Architecture Overview
4. Step 1: Prerequisites
5. Step 2: appsettings.json Configuration
6. Step 3: Create EmployeeEvent Model
7. Step 4: Create RabbitMQPublisher Service
8. Step 5: Register Publisher in Program.cs
9. Step 6: Update EmployeeService (Publish Events)
10. Step 7: Create EmployeeEventConsumer (Hosted Service)
11. Step 8: Register Hosted Service in Program.cs
12. Step 9: Test the Complete Flow



Key Concepts Explained

What is a Message Broker?

A **Message Broker** is a middleman between two services. Instead of Service A calling Service B directly, Service A sends a message to the broker, and Service B picks it up when ready.

Analogy: Think of it like a **post office**:

- The sender (Publisher) drops a letter in the mailbox
- The post office (RabbitMQ) stores and routes the letter
- The receiver (Consumer) picks it up when available
- The sender doesn't wait for the receiver — they're **decoupled**

What is RabbitMQ?

RabbitMQ is one of the most popular open-source message brokers. Key terms:

Term	Meaning	Analogy
Producer/Publisher	Sends messages to RabbitMQ	Person dropping a letter
Queue	A buffer where messages wait to be consumed	Mailbox
Consumer	Reads and processes messages from the queue	Postman delivering letters

Exchange	Routes messages to the right queue	Post office sorting department
Binding	Connection between exchange and queue	Delivery route
Channel	Lightweight connection inside the main connection	A lane in a highway

What is a Hosted Service?

A **Hosted Service** (BackgroundService) in .NET is a class that runs in the **background** for the lifetime of the application.

Analogy: Think of it like a **security guard** who works 24/7 in the background — always listening, always ready to act, never blocking the main entrance (the API).

Feature	Description
Runs automatically	Starts when app starts, stops when app stops
Non-blocking	Runs on a separate thread — doesn't affect API performance
Continuous	Can listen to queues, process jobs, run timers indefinitely
Base class	Inherit from BackgroundService and override ExecuteAsync

Why combine RabbitMQ + Hosted Service?

The **Hosted Service IS the Consumer**. It runs in the background, continuously listening to the RabbitMQ queue. When a message arrives, it wakes up and processes it — all without any impact on the main API.

The Demo Scenario

What we are building:

When an **Employee is Created or Deleted** via the API:

1. The EmployeeService publishes an **EmployeeEvent** message to RabbitMQ
2. The message travels through RabbitMQ and sits in a queue
3. The **Hosted Service (Consumer)** picks up the message
4. The consumer **logs the event** — simulating an audit trail or notification

This demonstrates **event-driven architecture** — the API doesn't wait for the consumer to finish. It fires the event and moves on.

Architecture Overview

```
HTTP Request (POST /api/employees) ↓ EmployeeController.Create() ↓ EmployeeService.AddAsync()  
_____ ↓ | publishes event Returns EmployeeDto to client (API responds immediately) ↓
```

RabbitMQPublisher ↓ RabbitMQ Queue "employee.events.Queue" ↓ EmployeeEventConsumer (BackgroundService) ↓ Processes message:
Logs "Employee John Created"

💡 **Key Point:** The API returns a response to the client **immediately** after saving the employee to the database. The RabbitMQ publishing happens, but the API doesn't wait for the consumer to process it. This is **asynchronous, non-blocking communication**.

① Step 1: Prerequisites

1a. Verify RabbitMQ is Running

Open browser and navigate to:

`http://localhost:15672`

Login with: **Username:** guest **Password:** guest

You should see the RabbitMQ Management Dashboard. Note the **Queues** tab — it will be empty now but we'll see our queue appear after implementation.

1b. Install RabbitMQ.Client NuGet Package

In Package Manager Console:

```
Install-Package RabbitMQ.Client -ProjectName EmployeeApp.Api
```

Or via NuGet Package Manager UI → search **RabbitMQ.Client** → install in **EmployeeApp.Api**.

② Step 2: appsettings.json Configuration

Purpose: Store RabbitMQ connection settings in config — not hardcoded in code. This makes it easy to change for different environments (Dev, Staging, Production).

File: EmployeeApp.Api/appsettings.json

```
{
  "RabbitMq": {
    "HostName": "localhost",
    "Port": 5672,
    "UserName": "guest",
    "Password": "guest",
    "VirtualHost": "/",
    "EmployeeQueue": "employee.events.Queue"
  }
}
```

Line-by-Line Explanation:

- ▶ **HostName: "localhost"** — RabbitMQ server address. Use the server IP/hostname in production.
- ▶ **Port: 5672** — Default AMQP port RabbitMQ listens on. (15672 is only for the management UI)
- ▶ **UserName / Password: "guest"** — Default credentials. Change in production!

- ▶ **VirtualHost:** "/" — RabbitMQ supports multiple virtual hosts for isolation. "/" is the default.
 - ▶ **EmployeeQueue:** "employee.events.Queue" — The name of the queue we'll create and use.
-

③ Step 3: Create EmployeeEvent Model

Purpose: Define the shape of the message that travels through RabbitMQ. This is the data contract between the Publisher and Consumer.

File: EmployeeApp.Api/Events/EmployeeEvent.cs

```
namespace EmployeeApp.Api.Events
{
    public class EmployeeEvent
    {
        public string EventType { get; set; }    // "Created" or "Deleted"
        public int    EmployeeId { get; set; }    // ID of the employee
        public string EmployeeName { get; set; }  // Name for human-readable log
        public DateTime OccurredAt { get; set; } // When the event happened
    }
}
```

Explanation:

- ▶ **EventType** — Tells the consumer WHAT happened. Examples: "Created", "Deleted", "Updated"
- ▶ **EmployeeId** — Identifies WHICH employee the event is about
- ▶ **EmployeeName** — Included for readable log messages without extra DB lookups
- ▶ **OccurredAt** — Timestamp for audit trail purposes

💡 Think of EmployeeEvent as the **letter inside the envelope** — it's what the consumer reads after opening the message from the queue.

④ Step 4: Create RabbitMQPublisher Service

Purpose: A service responsible for connecting to RabbitMQ and sending messages. The EmployeeService will use this to publish events.

File: EmployeeApp.Api/Messaging/RabbitMQPublisher.cs

```
using EmployeeApp.Api.Events;
using RabbitMQ.Client;
using System.Text;
using System.Text.Json;

namespace EmployeeApp.Api.Messaging
{
    public class RabbitMQPublisher : IDisposable
    {
        private readonly IConnection _connection;
        private readonly IChannel _channel;
        private readonly string _queueName;

        public RabbitMQPublisher(IConfiguration config)
        {
            var rabbitConfig = config.GetSection("RabbitMq");

            // Step A: Configure the connection settings
            var factory = new ConnectionFactory
            {
```



```

        HostName      = rabbitConfig["HostName"]!,
        Port          = int.Parse(rabbitConfig["Port"]!),
        UserName      = rabbitConfig["UserName"]!,
        Password       = rabbitConfig["Password"]!,
        VirtualHost    = rabbitConfig["VirtualHost"]!
    };

    _queueName = rabbitConfig["EmployeeQueue"]!;

    // Step B: Open the connection and channel (synchronous in constructor)
    _connection = factory.CreateConnectionAsync().GetAwaiter().GetResult();
    _channel    = _connection.CreateChannelAsync().GetAwaiter().GetResult();

    // Step C: Declare the queue – creates it in RabbitMQ if it doesn't exist
    _channel.QueueDeclareAsync(
        queue:      _queueName,
        durable:    true,
        exclusive:  false,
        autoDelete: false,
        arguments:  null
    ).GetAwaiter().GetResult();
}

// Called by EmployeeService when an employee is created or deleted
public async Task PublishAsync(EmployeeEvent employeeEvent)
{
    // Serialize the event object to JSON string
    var message = JsonSerializer.Serialize(employeeEvent);

    // Convert JSON string to bytes (RabbitMQ works with raw bytes)
    var body = Encoding.UTF8.GetBytes(message);

    // Mark message as persistent – survives RabbitMQ restart
    var properties = new BasicProperties { Persistent = true };

    // Publish the message to the queue
    await _channel.BasicPublishAsync(
        exchange:      string.Empty,
        routingKey:     _queueName,
        mandatory:      false,
        basicProperties: properties,
        body:           body
    );
}

```

```

        );
    }

    // Properly close connections when service is disposed
    public void Dispose()
    {
        _channel?.CloseAsync();
        _connection?.CloseAsync();
    }
}

```

Key Concept — Why GetAwaiter().GetResult() in constructor?

⚠ C# constructors **cannot be async**. They have no return type, so there's nowhere to return a Task.

GetAwaiter().GetResult() is the workaround — it **blocks the thread synchronously** until the async operation completes.

This is acceptable here because:

- It runs only **once at application startup**
- The connection setup is **very fast**
- Without the connection, the object cannot function at all

QueueDeclareAsync Parameters Explained:

Parameter	Value	Meaning
queue	"employee.events.Queue"	The name of the queue
durable	true	Queue survives RabbitMQ server restart

exclusive	false	Queue can be used by multiple connections
autoDelete	false	Queue does NOT delete itself when consumers disconnect
arguments	null	No extra configuration needed

PublishAsync Flow:

Step	Code	What happens
1	JsonSerializer.Serialize	EmployeeEvent object → JSON string
2	Encoding.UTF8.GetBytes	JSON string → byte array
3	BasicProperties { Persistent=true }	Marks message as durable
4	BasicPublishAsync	Sends bytes to the queue

⑤ Step 5: Register Publisher in Program.cs

Purpose: Register RabbitMQPublisher in the DI container so it can be injected into EmployeeService.

File: EmployeeApp.Api/Program.cs — add this line:

```
// Register RabbitMQPublisher as Singleton – one instance shared for whole app lifetime
builder.Services.AddSingleton<RabbitMQPublisher>();
```

💡 Why Singleton?

- Opening a RabbitMQ connection is **expensive** — we want to do it once
- A single connection with multiple channels is the recommended RabbitMQ pattern
- Scoped would create a new connection per HTTP request — wasteful

⑥ Step 6: Update EmployeeService to Publish Events

Purpose: After creating or deleting an employee, publish an event to RabbitMQ. The EmployeeService knows what happened — it's the right place to trigger the event.

File: EmployeeApp.Api/Services/EmployeeService.cs

```
using EmployeeApp.Api.Events;
using EmployeeApp.Api.Messaging;

public class EmployeeService(IEmployeeRepository repository, IMapper mapper,
    RabbitMQPublisher publisher) : IEmployeeService
{
    public async Task<EmployeeDto> AddAsync(EmployeeDto entity)
    {
        var employee      = mapper.Map<Employee>(entity);
        var savedEntity    = await repository.CreateAsync(employee);
        var result         = mapper.Map<EmployeeDto>(savedEntity);
```

```

        // Publish "Created" event to RabbitMQ after saving to DB
        await publisher.PublishAsync(new EmployeeEvent
        {
            EventType      = "Created",
            EmployeeId      = result.Id,
            EmployeeName    = result.Name,
            OccurredAt      = DateTime.UtcNow
        });

        return result;
    }

    public async Task<EmployeeDto> DeleteAsync(int id)
    {
        var deleted = await repository.DeleteAsync(id);
        var result  = mapper.Map<EmployeeDto>(deleted);

        // Publish "Deleted" event to RabbitMQ after deleting from DB
        await publisher.PublishAsync(new EmployeeEvent
        {
            EventType      = "Deleted",
            EmployeeId      = result.Id,
            EmployeeName    = result.Name,
            OccurredAt      = DateTime.UtcNow
        });

        return result;
    }

    // GetAllAsync and UpdateAsync remain unchanged...
}

```

What changed:

- ▶ `RabbitMQPublisher publisher` — injected via constructor (Primary Constructor syntax)
- ▶ `await publisher.PublishAsync(...)` — fires the event AFTER the DB operation succeeds

💡 **Important:** We publish AFTER the database operation. If the DB save fails, no event is published. This ensures data consistency — we only notify when something actually happened.

⑦ Step 7: Create EmployeeEventConsumer (Hosted Service)

Purpose: A background service that runs for the entire lifetime of the application, continuously listening to the RabbitMQ queue and processing messages as they arrive.

File: EmployeeApp.Api/Messaging/EmployeeEventConsumer.cs

```
using EmployeeApp.Api.Events;
using RabbitMQ.Client;
using RabbitMQ.Client.Events;
using System.Text;
using System.Text.Json;

namespace EmployeeApp.Api.Messaging
{
    // BackgroundService is the base class for Hosted Services in .NET
    public class EmployeeEventConsumer : BackgroundService
    {
        private readonly ILogger<EmployeeEventConsumer> _logger;
        private readonly IConfiguration _config;
        private IConnection? _connection;
        private IChannel? _channel;

        public EmployeeEventConsumer(
            ILogger<EmployeeEventConsumer> logger,
            IConfiguration config)
        {
            _logger = logger;
        }
    }
}
```

```

        _config = config;
    }

    // Called once when the Hosted Service starts
    public override async Task StartAsync(CancellationToken cancellationToken)
    {
        var rabbitConfig = _config.GetSection("RabbitMq");

        var factory = new ConnectionFactory
        {
            HostName      = rabbitConfig["HostName"]!,
            Port          = int.Parse(rabbitConfig["Port"]!),
            UserName      = rabbitConfig["UserName"]!,
            Password      = rabbitConfig["Password"]!,
            VirtualHost    = rabbitConfig["VirtualHost"]!
        };

        _connection = await factory.CreateConnectionAsync(cancellationToken);
        _channel     = await _connection.CreateChannelAsync(null, cancellationToken);

        _logger.LogInformation("EmployeeEventConsumer started. Listening to queue...");

        // Call the base StartAsync which then calls ExecuteAsync
        await base.StartAsync(cancellationToken);
    }

    // The main loop – runs continuously in the background
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        var queueName = _config.GetSection("RabbitMq")["EmployeeQueue"]!;

        // Declare the same queue – safe to declare multiple times, RabbitMQ ignores if exists
        await _channel!.QueueDeclareAsync(
            queue:      queueName,
            durable:    true,
            exclusive:  false,
            autoDelete: false,
            arguments:  null,
            cancellationToken: stoppingToken);

        // Create a consumer that handles incoming messages
        var consumer = new AsyncEventingBasicConsumer(_channel);
    }

```

```

// This event fires every time a new message arrives in the queue
consumer.ReceivedAsync += async (sender, eventArgs) =>
{
    // Step 1: Read the raw bytes from the message
    var body = eventArgs.Body.ToArray();

    // Step 2: Convert bytes back to JSON string
    var message = Encoding.UTF8.GetString(body);

    // Step 3: Deserialize JSON string back to EmployeeEvent object
    var employeeEvent = JsonSerializer.Deserialize<EmployeeEvent>(message,
        new JsonSerializerOptions { PropertyNameCaseInsensitive = true });

    if (employeeEvent != null)
    {
        // Step 4: Process the event – here we just log it
        _logger.LogInformation(
            "[AUDIT] Employee {EventType}: ID={Id}, Name={Name}, Time={Time}",
            employeeEvent.EventType,
            employeeEvent.EmployeeId,
            employeeEvent.EmployeeName,
            employeeEvent.OccurredAt);
    }

    // Step 5: Acknowledge – tell RabbitMQ "I processed this, remove it from queue"
    await _channel.BasicAckAsync(eventArgs.DeliveryTag, multiple: false);
};

// Start consuming – tell RabbitMQ to send messages to this consumer
await _channel.BasicConsumeAsync(
    queue:      queueName,
    autoAck:    false, // We manually acknowledge – more reliable
    consumer:   consumer,
    cancellationToken: stoppingToken);

// Keep running until the app stops (stoppingToken is cancelled)
await Task.Delay(Timeout.Infinite, stoppingToken);
}

// Called when the app shuts down – clean up connections
public override async Task StopAsync(CancellationToken cancellationToken)

```



```

    {
        _logger.LogInformation("EmployeeEventConsumer stopping...");
        if (_channel != null) await _channel.CloseAsync();
        if (_connection != null) await _connection.CloseAsync();
        await base.StopAsync(cancellationToken);
    }
}

```

BackgroundService Lifecycle:

Method	When Called	What We Do
StartAsync	Once when app starts	Connect to RabbitMQ, set up channel
ExecuteAsync	Runs continuously	Listen to queue, process messages forever
StopAsync	Once when app shuts down	Close connections cleanly

Message Processing Flow:

Step	Code	What Happens
1	eventArgs.Body.ToArray()	Get raw bytes from the message
2	Encoding.UTF8.GetString(body)	Bytes → JSON string
3	JsonSerializer.Deserialize	JSON string → EmployeeEvent object
4	_logger.LogInformation	Write to application log (audit trail)

5	BasicAckAsync	Tell RabbitMQ: "Done, remove from queue"
---	---------------	--

💡 What is Acknowledgement (Ack)?

By default, when a consumer receives a message, RabbitMQ keeps it in the queue until the consumer sends an **Ack (acknowledgement)**. This is important:

- If the consumer crashes BEFORE acking → RabbitMQ re-delivers the message to another consumer
- If `autoAck=true` → message is removed as soon as it's received (risky — message lost if processing fails)
- If `autoAck=false` (our choice) → we manually ack AFTER processing → **safer and more reliable**

⑧ Step 8: Register Hosted Service in Program.cs

Purpose: Tell ASP.NET Core to run `EmployeeEventConsumer` as a background service automatically when the application starts.

File: `EmployeeApp.Api/Program.cs` — add this line:

```
// Register EmployeeEventConsumer as a Hosted Service
builder.Services.AddHostedService<EmployeeEventConsumer>();

// Also register the publisher as Singleton
builder.Services.AddSingleton<RabbitMQPublisher>();
```

AddHostedService vs AddSingleton:

Method	Use For	Difference
AddHostedService<T>()	Background services (Consumer)	Auto-starts when app starts, stops when app stops
AddSingleton<T>()	Regular services (Publisher)	Created on first injection, shared for app lifetime

⑨ Step 9: Test the Complete Flow

Purpose: Verify the end-to-end flow — create/delete an employee and see the message travel through RabbitMQ to the consumer.

Test Checklist:

1. Start the API and watch the console output

You should see on startup:

```
info: EmployeeApp.Api.Messaging.EmployeeEventConsumer[0]  
      EmployeeEventConsumer started. Listening to queue...
```

2. Check RabbitMQ Management UI

Open **http://localhost:15672** → Queues tab

You should see **"employee.events.Queue"** listed 

3. Create an Employee (via Swagger or Postman)

```
POST https://localhost:7021/api/employees
```

```
Authorization: Bearer <your-token>
```

```
Content-Type: application/json
```

```
{
  "name": "John Smith",
  "gender": "Male",
  "age": 30,
  "salary": 50000,
  "deptId": 1
}
```

4. Watch the API Console

You should immediately see the consumer log:

```
info: EmployeeApp.Api.Messaging.EmployeeEventConsumer[0]
      [AUDIT] Employee Created: ID=5, Name=John Smith, Time=2025-07-01T10:30:00
```

5. Delete the Employee

```
DELETE https://localhost:7021/api/employees/5
```

```
Authorization: Bearer <your-token>
```

6. Watch the console again:

```
info: EmployeeApp.Api.Messaging.EmployeeEventConsumer[0]
      [AUDIT] Employee Deleted: ID=5, Name=John Smith, Time=2025-07-01T10:31:00
```

✓ Summary — What We Built

Step	What We Created	File
1	Install RabbitMQ.Client NuGet package	EmployeeApp.Api.csproj
2	RabbitMQ configuration	appsettings.json
3	EmployeeEvent model (message contract)	Events/EmployeeEvent.cs
4	RabbitMQPublisher (sends messages)	Messaging/RabbitMQPublisher.cs
5	Register Publisher in DI	Program.cs
6	Publish events from EmployeeService	Services/EmployeeService.cs
7	EmployeeEventConsumer (BackgroundService)	Messaging/EmployeeEventConsumer.cs
8	Register Hosted Service in DI	Program.cs
9	Test the complete flow	Browser / Postman

Key Concepts Demonstrated to Trainees:

Concept	Where Demonstrated
Message Broker pattern	RabbitMQ queue between Publisher and Consumer
Publisher / Producer	RabbitMQPublisher.cs

Consumer	EmployeeEventConsumer.cs
Hosted Service / BackgroundService	EmployeeEventConsumer extends BackgroundService
Service lifetime (Singleton)	RabbitMQPublisher registered as Singleton
Manual Acknowledgement	BasicAckAsync in consumer
Event-driven architecture	API responds immediately, consumer processes asynchronously
Durable queues & messages	durable:true and Persistent:true
Decoupled services	EmployeeService doesn't know about the consumer